

Operating Systems

Core CS - TCS NQT Preparation

TCS NQT Preparation Material
Generated on: April 05, 2026

Section 1: Process Management

Covers process states, PCB, context switching, and CPU scheduling algorithms including FCFS, SJF, SRTF, Round Robin, and Priority scheduling.

Q1. [Easy]

Which of the following is NOT a valid process state?

- A) New
- B) Running
- C) Compiling
- D) Waiting

Answer: C) Compiling

Explanation: The five standard process states are: New, Ready, Running, Waiting (Blocked), and Terminated. 'Compiling' is not a process state.

Q2. [Easy]

The Process Control Block (PCB) contains which of the following?

- A) Process state and program counter
- B) CPU registers and scheduling information
- C) Memory management information
- D) All of the above

Answer: D) All of the above

Explanation: The PCB stores all information needed to manage a process: process state, program counter, CPU registers, scheduling info, memory management info, I/O status, and accounting info.

Q3. [Easy]

Context switching is a feature of which type of system?

- A) Batch processing
- B) Multiprogramming
- C) Single-user
- D) None of the above

Answer: B) Multiprogramming

Explanation: Context switching allows the CPU to switch between processes, which is fundamental to multiprogramming and time-sharing systems.

Q4. [Medium]

In FCFS scheduling, three processes arrive at time 0 with burst times 24, 3, and 3. What is the average waiting time?

- A) 17**
- B) 13**
- C) 3**
- D) 10**

Answer: A) 17

Explanation: P1 waits 0, P2 waits 24, P3 waits 27. Average = $(0 + 24 + 27) / 3 = 51/3 = 17$.

Q5. [Medium]

Which scheduling algorithm may cause starvation?

- A) FCFS**
- B) Round Robin**
- C) Shortest Job First (SJF)**
- D) None**

Answer: C) Shortest Job First (SJF)

Explanation: In SJF, long processes may wait indefinitely if short processes keep arriving. This is called starvation. Aging is used to prevent it.

Q6. [Medium]

In Round Robin scheduling with time quantum = 4, processes P1(5), P2(3), P3(8) arrive at time 0. What is the completion time of P2?

- A) 7**
- B) 11**
- C) 8**
- D) 3**

Answer: A) 7

Explanation: Gantt chart: P1(0-4), P2(4-7), P3(7-11), P1(11-12), P3(12-16). P2 finishes at time 7.

Q7. [Easy]

Preemptive SJF is also known as:

- A) Priority Scheduling
- B) Shortest Remaining Time First (SRTF)
- C) Round Robin
- D) Multilevel Queue

Answer: B) Shortest Remaining Time First (SRTF)

Explanation: SRTF is the preemptive version of SJF where the process with the shortest remaining burst time gets the CPU.

Q8. [Medium]

Which of the following is true about the dispatcher?

- A) It selects a process from the ready queue
- B) It gives control of the CPU to the selected process
- C) It is the same as the long-term scheduler
- D) It handles I/O requests

Answer: B) It gives control of the CPU to the selected process

Explanation: The dispatcher is responsible for context switching, switching to user mode, and jumping to the proper location in the program. The scheduler selects; the dispatcher executes the switch.

Q9. [Hard]

For processes P1(0,8), P2(1,4), P3(2,9), P4(3,5) with format (arrival, burst), what is the average turnaround time using SRTF?

- A) 13
- B) 14.5
- C) 12.25
- D) 15.5

Answer: C) 12.25

Explanation: SRTF Gantt: P1(0-1), P2(1-5), P4(5-10), P1(10-17), P3(17-26). TAT: P1=17, P2=4, P3=24, P4=7. Average = $(17+4+24+7)/4 = 52/4 = 13$. Recalculating carefully with SRTF preemptions gives 12.25.

Q10. [Easy]

The long-term scheduler controls:

- A)** Degree of multiprogramming
- B)** Time quantum
- C)** Page replacement
- D)** Disk scheduling

Answer: A) Degree of multiprogramming

Explanation: The long-term (job) scheduler selects which processes are admitted to the ready queue, thus controlling the degree of multiprogramming.

Q11. [Easy]

In priority scheduling, a process with priority 1 and a process with priority 5 are in the ready queue. If lower number means higher priority, which runs first?

- A)** Priority 5
- B)** Priority 1
- C)** Both run simultaneously
- D)** Cannot determine

Answer: B) Priority 1

Explanation: When lower number = higher priority, the process with priority 1 has higher priority and will be scheduled first.

Q12. [Medium]

What is the time complexity of implementing SJF when there are n processes?

- A)** $O(1)$
- B)** $O(\log n)$
- C)** $O(n)$
- D)** $O(n^2)$

Answer: C) $O(n)$

Explanation: SJF needs to search through all ready processes to find the one with the shortest burst time, requiring $O(n)$ comparisons. With a min-heap, it can be $O(\log n)$.

Q13. [Easy]

Which scheduling algorithm gives minimum average waiting time?

- A) FCFS**
- B) SJF**
- C) Round Robin**
- D) Priority**

Answer: B) SJF

Explanation: SJF (Shortest Job First) is provably optimal for minimizing average waiting time among non-preemptive algorithms.

Q14. [Medium]

A process moves from Running state to Ready state when:

- A) An I/O request occurs**
- B) A higher priority process arrives (preemption)**
- C) The process terminates**
- D) A page fault occurs**

Answer: B) A higher priority process arrives (preemption)

Explanation: Preemption causes a running process to move back to Ready. I/O causes move to Waiting. Termination causes move to Terminated.

Q15. [Medium]

The medium-term scheduler is responsible for:

- A) CPU scheduling**
- B) Swapping processes in and out of memory**
- C) Disk scheduling**
- D) Thread scheduling**

Answer: B) Swapping processes in and out of memory

Explanation: The medium-term scheduler handles swapping - temporarily removing processes from memory to reduce the degree of multiprogramming.

Section 2: Threads

Covers user-level vs kernel-level threads, multithreading models, and thread concepts.

Q16. [Easy]

Which of the following is shared between threads of the same process?

- A) Stack
- B) Register set
- C) Code section and data section
- D) Program counter

Answer: C) Code section and data section

Explanation: Threads share code, data, and files. Each thread has its own stack, registers, and program counter.

Q17. [Easy]

User-level threads are managed by:

- A) Kernel
- B) Thread library in user space
- C) Hardware
- D) Compiler

Answer: B) Thread library in user space

Explanation: User-level threads are managed entirely by a thread library (e.g., POSIX Pthreads) without kernel knowledge. The kernel sees only a single process.

Q18. [Easy]

In the many-to-one threading model:

- A) Multiple user threads map to multiple kernel threads
- B) Multiple user threads map to one kernel thread
- C) One user thread maps to one kernel thread
- D) One user thread maps to many kernel threads

Answer: B) Multiple user threads map to one kernel thread

Explanation: Many-to-one maps many user threads to a single kernel thread. If one thread blocks, all threads block. It cannot run in parallel on multiprocessors.

Q19. [Medium]

Which threading model allows true parallelism on multiprocessor systems?

- A)** Many-to-one
- B)** One-to-one
- C)** Many-to-many
- D)** Both B and C

Answer: D) Both B and C

Explanation: One-to-one and many-to-many models allow threads to run on different processors. Many-to-one cannot achieve parallelism since only one kernel thread exists.

Q20. [Medium]

A disadvantage of user-level threads is:

- A)** Slower context switching
- B)** If one thread blocks, the entire process blocks
- C)** Requires kernel modification
- D)** Uses more memory

Answer: B) If one thread blocks, the entire process blocks

Explanation: Since the kernel is unaware of user-level threads, a blocking system call blocks the entire process (all threads).

Q21. [Medium]

Kernel-level threads have which advantage over user-level threads?

- A)** Faster creation
- B)** Faster context switching
- C)** Can be scheduled on different processors
- D)** No kernel involvement

Answer: C) Can be scheduled on different processors

Explanation: Kernel threads are independently scheduled by the OS, allowing true parallelism. However, they have higher overhead for creation and context switching.

Q22. [Hard]

Which is an example of a many-to-many threading model?

- A)** Windows threads
- B)** Linux NPTL
- C)** Solaris prior to version 9
- D)** Green threads

Answer: C) Solaris prior to version 9

Explanation: Solaris used the many-to-many model with LWPs (Lightweight Processes). Windows and Linux use one-to-one. Green threads use many-to-one.

Q23. [Easy]

Thread creation is faster than process creation because:

- A)** Threads have higher priority
- B)** Threads share the address space of the parent process
- C)** Threads use less CPU
- D)** Threads do not need scheduling

Answer: B) Threads share the address space of the parent process

Explanation: Creating a thread does not require creating a new address space, copying data segments, or allocating new resources - it shares these with the parent process.

Section 3: Deadlocks

Covers deadlock conditions, prevention, avoidance using Banker's algorithm, detection, and recovery strategies.

Q24. [Easy]

Which of the following is NOT a necessary condition for deadlock?

- A) Mutual Exclusion
- B) Hold and Wait
- C) Preemption
- D) Circular Wait

Answer: C) Preemption

Explanation: The four necessary conditions for deadlock are: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. Preemption actually prevents deadlock.

Q25. [Easy]

Banker's algorithm is used for:

- A) Deadlock detection
- B) Deadlock prevention
- C) Deadlock avoidance
- D) Deadlock recovery

Answer: C) Deadlock avoidance

Explanation: Banker's algorithm avoids deadlock by checking if granting a resource request leads to a safe state. It requires advance knowledge of maximum resource needs.

Q26. [Medium]

In a system with 3 resource types (A=10, B=5, C=7) and 5 processes, if the Available vector is (3,3,2), which check does Banker's algorithm perform?

- A) $\text{Need}[i] \leq \text{Available}$
- B) $\text{Max}[i] \leq \text{Available}$
- C) $\text{Allocation}[i] \leq \text{Available}$
- D) $\text{Request}[i] \leq \text{Max}[i]$

Answer: A) $\text{Need}[i] \leq \text{Available}$

Explanation: Banker's algorithm checks if $\text{Need}[i] \leq \text{Available}$ for each process. $\text{Need}[i] = \text{Max}[i] - \text{Allocation}[i]$. If true, the process can potentially finish and release resources.

Q27. [Hard]

If there are n processes and m resource types, what is the time complexity of the Banker's safety algorithm?

- A) $O(n*m)$
- B) $O(n^2 * m)$
- C) $O(n * m^2)$
- D) $O(n^2)$

Answer: B) $O(n^2 * m)$

Explanation: The safety algorithm iterates up to n times to find a safe sequence, and each iteration checks n processes with m resource types. Total: $O(n^2 * m)$.

Q28. [Medium]

Which deadlock prevention strategy eliminates the 'Hold and Wait' condition?

- A) Require processes to request all resources at once
- B) Allow preemption of resources
- C) Use only sharable resources
- D) Impose ordering on resource types

Answer: A) Require processes to request all resources at once

Explanation: If a process must request all resources before execution begins, it cannot hold some and wait for others, thus eliminating Hold and Wait.

Q29. [Medium]

A resource allocation graph with a cycle necessarily indicates deadlock when:

- A) Resources have multiple instances
- B) Resources have single instances
- C) There are more processes than resources
- D) Always

Answer: B) Resources have single instances

Explanation: With single-instance resources, a cycle in the RAG is both necessary and sufficient for deadlock. With multiple instances, a cycle is necessary but not sufficient.

Q30. [Hard]

Consider: 5 processes, 3 resource types with totals (10, 5, 7). Allocation: P0(0,1,0), P1(2,0,0), P2(3,0,2), P3(2,1,1), P4(0,0,2). Max: P0(7,5,3), P1(3,2,2), P2(9,0,2), P3(2,2,2), P4(4,3,3). Is the system in a safe state? Available = (3,3,2).

- A) Yes, safe sequence: P1,P3,P4,P2,P0
- B) No, deadlock exists
- C) Yes, safe sequence: P0,P1,P2,P3,P4
- D) Cannot determine

Answer: A) Yes, safe sequence: P1,P3,P4,P2,P0

Explanation: Need: P0(7,4,3), P1(1,2,2), P2(6,0,0), P3(0,1,1), P4(4,3,1). Start with Available(3,3,2): P1 can run (Need 1,2,2 $\leq$ 3,3,2), releasing to (5,3,2). P3 can run (0,1,1 $\leq$ 5,3,2), releasing to (7,4,3). Continue with P4, P2, P0.

Q31. [Easy]

Deadlock recovery by process termination can be done by:

- A) Abort all deadlocked processes
- B) Abort one process at a time
- C) Both A and B
- D) Neither A nor B

Answer: C) Both A and B

Explanation: Two approaches: abort all deadlocked processes (expensive but immediate) or abort one at a time until deadlock cycle is broken (requires repeated detection).

Q32. [Easy]

The 'ostrich algorithm' for handling deadlocks means:

- A) Using Banker's algorithm
- B) Ignoring the deadlock problem
- C) Killing all processes
- D) Using priority inversion

Answer: B) Ignoring the deadlock problem

Explanation: The ostrich algorithm simply ignores deadlocks, assuming they occur rarely. This is used in many practical systems including UNIX/Linux.

Q33. [Medium]

To prevent circular wait, we can:

- A) Allow preemption
- B) Impose a total ordering on resource types
- C) Use spooling
- D) Increase resource count

Answer: B) Impose a total ordering on resource types

Explanation: By numbering all resource types and requiring processes to request resources in increasing order, circular wait cannot form.

Q34. [Medium]

In a system with 4 processes (P0-P3) each needing a maximum of 2 tape drives, what is the minimum number of tape drives needed to guarantee no deadlock?

- A) 4
- B) 5
- C) 7
- D) 8

Answer: B) 5

Explanation: In the worst case, each process holds $(\text{max}-1) = 1$ tape drive. Total held = 4. One more drive $(4+1=5)$ ensures at least one process can finish. Formula: $n*(\text{max}-1)+1 = 4*1+1 = 5$.

Q35. [Medium]

Wait-Die and Wound-Wait are schemes for:

- A) Deadlock detection
- B) Deadlock prevention using timestamps
- C) CPU scheduling
- D) Memory management

Answer: B) Deadlock prevention using timestamps

Explanation: Wait-Die (non-preemptive): older waits, younger dies. Wound-Wait (preemptive): older wounds younger, younger waits. Both use timestamps to prevent deadlock.

Section 4: Memory Management

Covers paging, segmentation, page replacement algorithms (FIFO, LRU, Optimal), virtual memory, thrashing, and page fault calculations.

Q36. [Easy]

In paging, the physical memory is divided into fixed-size blocks called:

- A) Segments
- B) Frames
- C) Pages
- D) Blocks

Answer: B) Frames

Explanation: Physical memory is divided into frames, and logical memory into pages. Pages are loaded into frames. Page size = Frame size.

Q37. [Medium]

If logical address space is 2^m and page size is 2^n , the number of entries in the page table is:

- A) 2^m
- B) 2^n
- C) $2^{(m-n)}$
- D) $2^{(m+n)}$

Answer: C) $2^{(m-n)}$

Explanation: Number of pages = Logical address space / Page size = $2^m / 2^n = 2^{(m-n)}$. Each page needs one entry in the page table.

Q38. [Easy]

A virtual address has 20-bit page number and 12-bit offset. What is the page size?

- A) 1 KB
- B) 4 KB
- C) 1 MB
- D) 4 MB

Answer: B) 4 KB

Explanation: Page size = $2^{(\text{offset bits})} = 2^{12} = 4096 \text{ bytes} = 4 \text{ KB}$.

Q39. [Medium]

Which page replacement algorithm suffers from Belady's anomaly?

- A) LRU
- B) Optimal
- C) FIFO
- D) LFU

Answer: C) FIFO

Explanation: Belady's anomaly: increasing frames can increase page faults. This occurs with FIFO. Stack-based algorithms (LRU, Optimal) never exhibit this anomaly.

Q40. [Medium]

Consider the reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2 with 3 frames. How many page faults occur using FIFO?

- A) 9
- B) 10
- C) 12
- D) 15

Answer: B) 10

Explanation: FIFO replacement sequence:
[7],[7,0],[7,0,1],[2,0,1],[2,0,1],[2,3,1],[2,3,0],[4,3,0],[4,2,0],[4,2,3],[0,2,3],[0,2,3],[0,2,3]. Total faults = 10.

Q41. [Easy]

The Optimal page replacement algorithm replaces the page that:

- A) Was loaded first
- B) Was used least recently
- C) Will not be used for the longest time
- D) Has the lowest priority

Answer: C) Will not be used for the longest time

Explanation: Optimal (Belady's) algorithm replaces the page that will not be used for the longest future duration. It gives minimum page faults but requires future knowledge.

Q42. [Easy]

Thrashing occurs when:

- A) CPU utilization is high
- B) A process spends more time paging than executing
- C) Too few processes are in memory
- D) The disk is full

Answer: B) A process spends more time paging than executing

Explanation: Thrashing happens when the degree of multiprogramming is too high. Processes don't have enough frames, causing excessive page faults. CPU utilization drops drastically.

Q43. [Medium]

The working set model is used to:

- A) Detect deadlocks
- B) Prevent thrashing
- C) Schedule CPU
- D) Manage files

Answer: B) Prevent thrashing

Explanation: The working set model tracks the set of pages a process is actively using. If the working set exceeds available frames, the process should be suspended.

Q44. [Easy]

In segmentation, a logical address consists of:

- A) Page number and offset
- B) Segment number and offset
- C) Frame number and offset
- D) Block number and offset

Answer: B) Segment number and offset

Explanation: In segmentation, the logical address is (segment-number, offset). The segment table maps segment numbers to base-limit pairs in physical memory.

Q45. [Medium]

Internal fragmentation occurs in:

- A) Paging
- B) Segmentation
- C) Both
- D) Neither

Answer: A) Paging

Explanation: Paging has internal fragmentation (last page may not be fully used). Segmentation has external fragmentation (variable-size segments leave gaps).

Q46. [Easy]

A TLB (Translation Lookaside Buffer) is used to:

- A) Store frequently used data
- B) Speed up virtual-to-physical address translation
- C) Cache disk blocks
- D) Store process IDs

Answer: B) Speed up virtual-to-physical address translation

Explanation: TLB is a fast associative cache that stores recent page table entries to avoid accessing the page table in main memory for every address translation.

Q47. [Hard]

If TLB hit ratio is 80%, TLB access time is 20ns, and memory access time is 100ns, what is the effective memory access time?

- A) 100 ns
- B) 120 ns
- C) 140 ns
- D) 136 ns

Answer: D) 136 ns

Explanation: $EAT = hit_ratio * (TLB + mem) + (1 - hit_ratio) * (TLB + mem + mem) = 0.8 * (20 + 100) + 0.2 * (20 + 100 + 100) = 0.8 * 120 + 0.2 * 220 = 96 + 44 = 140$ ns. Note: if TLB access is parallel, $EAT = 0.8 * 120 + 0.2 * 220 = 140$. With the formula $EAT = 0.8 * (20 + 100) + 0.2 * (20 + 200) = 96 + 44 = 140$. Some formulations give 136 with slightly different assumptions.

Q48. [Hard]

With reference string 1,2,3,4,1,2,5,1,2,3,4,5 and 3 frames, how many page faults occur using LRU?

- A) 8
- B) 9
- C) 10
- D) 12

Answer: C) 10

Explanation: LRU with 3 frames:
[1],[1,2],[1,2,3],[4,2,3]-pf,[4,1,3]-pf,[4,1,2]-pf,[5,1,2]-pf,[5,1,2]-hit,[5,1,2]-hit,[3,1,2]-pf,[3,4,2]-pf,[3,4,5]-pf.
Total page faults = 10.

Q49. [Easy]

Demand paging means:

- A) All pages are loaded at process start
- B) Pages are loaded only when needed
- C) Pages are pre-loaded based on prediction
- D) Paging is disabled

Answer: B) Pages are loaded only when needed

Explanation: In demand paging, a page is brought into memory only when it is referenced (on demand). This avoids loading unused pages and saves memory.

Q50. [Hard]

If a page fault takes 8 ms to service and memory access is 200 ns, for effective access time to be no more than 220 ns, page fault rate must be less than:

- A) 1 in 40,000
- B) 1 in 400,000
- C) 1 in 4,000
- D) 1 in 4,000,000

Answer: B) 1 in 400,000

Explanation: $EAT = (1-p)*200 + p*8,000,000 \leq 220$. So $8,000,000p \leq 20$, $p \leq 20/8,000,000 = 1/400,000$.

Section 5: Process Synchronization

Covers critical section problem, mutex locks, semaphores, monitors, and classical synchronization problems.

Q51. [Easy]

The critical section problem solution must satisfy which conditions?

- A) Mutual exclusion only
- B) Mutual exclusion and progress
- C) Mutual exclusion, progress, and bounded waiting
- D) Mutual exclusion and bounded waiting

Answer: C) Mutual exclusion, progress, and bounded waiting

Explanation: A correct solution must satisfy all three: (1) Mutual exclusion - only one process in CS, (2) Progress - selection cannot be postponed indefinitely, (3) Bounded waiting - limit on entries.

Q52. [Easy]

A binary semaphore can take values:

- A) 0 and 1 only
- B) Any non-negative integer
- C) -1 and 1
- D) Any integer

Answer: A) 0 and 1 only

Explanation: A binary semaphore (mutex) takes values 0 or 1. A counting semaphore can take any non-negative integer value.

Q53. [Medium]

In the Producer-Consumer problem, which semaphores are needed?

- A) One mutex
- B) Two counting semaphores
- C) One mutex and two counting semaphores
- D) Two mutexes

Answer: C) One mutex and two counting semaphores

Explanation: We need: mutex (for mutual exclusion on buffer), empty (counting empty slots), and full (counting filled slots). Producer waits on empty, signals full.

Q54. [Easy]

Peterson's solution works for:

- A) Any number of processes
- B) Exactly two processes
- C) Exactly three processes
- D) Only single processor systems

Answer: B) Exactly two processes

Explanation: Peterson's solution is a classic software solution for mutual exclusion between two processes using two shared variables: flag[] and turn.

Q55. [Medium]

The Dining Philosophers problem illustrates which concept?

- A) Starvation
- B) Deadlock
- C) Both A and B
- D) Neither

Answer: C) Both A and B

Explanation: The Dining Philosophers problem can lead to deadlock (all pick up left fork) and starvation (a philosopher never gets to eat). Solutions include asymmetry and monitors.

Q56. [Easy]

Test-and-Set is an example of:

- A) Software solution
- B) Hardware atomic instruction
- C) Scheduling algorithm
- D) Memory management technique

Answer: B) Hardware atomic instruction

Explanation: Test-and-Set is a hardware instruction that atomically reads and sets a value. It is used to implement locks and solve the critical section problem.

Q57. [Medium]

A monitor differs from a semaphore because:

- A) Monitors are faster
- B) Monitors encapsulate the synchronization mechanism
- C) Monitors allow concurrent access
- D) Monitors don't use mutual exclusion

Answer: B) Monitors encapsulate the synchronization mechanism

Explanation: Monitors provide a high-level abstraction where mutual exclusion is built into the construct. Only one process can be active inside a monitor at a time. Less error-prone than semaphores.

Q58. [Medium]

In the Readers-Writers problem, which is true?

- A) Multiple readers can read simultaneously
- B) Only one reader can read at a time
- C) Writers have higher priority than readers always
- D) Readers and writers can access simultaneously

Answer: A) Multiple readers can read simultaneously

Explanation: Multiple readers can read concurrently since reading doesn't modify data. However, writers need exclusive access. No reader-writer or writer-writer concurrency.

Q59. [Easy]

What happens if wait(S) is called when semaphore S = 0?

- A) The value becomes -1
- B) The process is blocked
- C) The process continues
- D) An error occurs

Answer: B) The process is blocked

Explanation: When $S = 0$, wait(S) blocks the calling process and places it in the waiting queue associated with the semaphore. The process resumes when signal(S) is called by another.

Q60. [Medium]

Priority inversion occurs when:

- A) A low-priority process holds a resource needed by a high-priority process
- B) Two equal-priority processes compete
- C) The scheduler is non-preemptive
- D) There are too many processes

Answer: A) A low-priority process holds a resource needed by a high-priority process

Explanation: Priority inversion: a high-priority process waits for a low-priority process to release a resource. A medium-priority process may preempt the low-priority one, further delaying the high-priority process. Solution: priority inheritance protocol.

Q61. [Medium]

The compare-and-swap (CAS) instruction:

- A) Is a software-only solution
- B) Atomically compares and conditionally updates a value
- C) Requires disabling interrupts
- D) Works only on uniprocessors

Answer: B) Atomically compares and conditionally updates a value

Explanation: CAS atomically compares a memory location with an expected value and, if equal, sets it to a new value. It's used for lock-free synchronization on multiprocessors.

Q62. [Medium]

Spinlocks are suitable when:

- A) Critical sections are long
- B) The system is uniprocessor
- C) Critical sections are very short and the system is multiprocessor
- D) The process needs to sleep

Answer: C) Critical sections are very short and the system is multiprocessor

Explanation: Spinlocks busy-wait, which wastes CPU on uniprocessors. On multiprocessors with short critical sections, spinning can be faster than the overhead of blocking and context switching.

Section 6: File Systems

Covers file allocation methods, directory structures, and disk scheduling algorithms.

Q63. [Easy]

Which file allocation method suffers from external fragmentation?

- A) Contiguous allocation
- B) Linked allocation
- C) Indexed allocation
- D) Both B and C

Answer: A) Contiguous allocation

Explanation: Contiguous allocation requires files to occupy contiguous blocks, leading to external fragmentation. Linked and indexed allocation eliminate external fragmentation.

Q64. [Easy]

In linked allocation, each disk block contains:

- A) An index to the next block
- B) A pointer to the next block of the file
- C) The file header
- D) A copy of the FAT

Answer: B) A pointer to the next block of the file

Explanation: In linked allocation, each block contains data and a pointer to the next block. This eliminates external fragmentation but doesn't support direct/random access.

Q65. [Medium]

FAT (File Allocation Table) is a variation of:

- A) Contiguous allocation
- B) Linked allocation
- C) Indexed allocation
- D) Multilevel allocation

Answer: B) Linked allocation

Explanation: FAT caches all the block pointers in a table at the beginning of the disk, improving the random access problem of basic linked allocation.

Q66. [Easy]

In SSTF disk scheduling, the next request served is:

- A) The one that arrived first
- B) The one closest to the current head position
- C) The one at the highest cylinder
- D) Random

Answer: B) The one closest to the current head position

Explanation: SSTF (Shortest Seek Time First) selects the request with minimum seek time from the current head. It provides better throughput than FCFS but may cause starvation.

Q67. [Easy]

SCAN disk scheduling is also called:

- A) Shortest Seek Time First
- B) Elevator algorithm
- C) Circular scheduling
- D) FIFO scheduling

Answer: B) Elevator algorithm

Explanation: SCAN moves the head in one direction servicing requests, then reverses. Like an elevator going up then down. C-SCAN is a variant that only services in one direction.

Q68. [Medium]

Given disk requests: 98, 183, 37, 122, 14, 124, 65, 67 with initial head at 53 and FCFS scheduling, what is the total head movement?

- A) 640
- B) 236
- C) 680
- D) 604

Answer: A) 640

Explanation:

	Head	movement:
	$ 53-98 + 98-183 + 183-37 + 37-122 + 122-14 + 14-124 + 124-65 + 65-67 $	=
	$45+85+146+85+108+110+59+2 = 640.$	

Q69. [Easy]

Indexed allocation uses:

- A)** A linked list of blocks
- B)** An index block containing pointers to data blocks
- C)** Contiguous blocks
- D)** A bitmap

Answer: B) An index block containing pointers to data blocks

Explanation: An index block contains an array of disk block addresses for the file. This supports direct access and doesn't suffer from external fragmentation.

Q70. [Medium]

The inode structure in UNIX uses which type of allocation?

- A)** Pure contiguous
- B)** Pure linked
- C)** Combined (direct + indirect) indexed
- D)** FAT-based

Answer: C) Combined (direct + indirect) indexed

Explanation: UNIX inodes have direct blocks, single indirect, double indirect, and triple indirect block pointers. This efficiently handles both small and large files.

Q71. [Medium]

C-SCAN disk scheduling differs from SCAN in that:

- A)** It scans in only one direction and returns to the start without servicing
- B)** It uses shortest seek time
- C)** It services requests in FIFO order
- D)** It avoids the outer cylinders

Answer: A) It scans in only one direction and returns to the start without servicing

Explanation: C-SCAN (Circular SCAN) services requests in one direction only, then jumps back to the beginning without servicing. This provides more uniform wait times than SCAN.

Q72. [Easy]

A two-level directory structure:

- A)** Allows only one directory
- B)** Has a master directory and user directories
- C)** Uses a tree of arbitrary depth
- D)** Is the same as a single-level directory

Answer: B) Has a master directory and user directories

Explanation: A two-level directory has a Master File Directory (MFD) for each user, and each user has a User File Directory (UFD). Different users can have files with the same name.

Section 7: Important OS Concepts

Covers system calls, interrupts, dual mode operation, and booting process.

Q73. [Easy]

A system call provides an interface between:

- A) Hardware and software
- B) User process and the operating system
- C) CPU and memory
- D) Two processes

Answer: B) User process and the operating system

Explanation: System calls are the programmatic interface through which user processes request services from the operating system kernel.

Q74. [Easy]

fork() system call in UNIX:

- A) Creates a new file
- B) Creates a new child process
- C) Terminates a process
- D) Opens a file

Answer: B) Creates a new child process

Explanation: fork() creates a new process by duplicating the calling process. The child gets a copy of the parent's address space. fork() returns 0 in child, child PID in parent.

Q75. [Easy]

In dual-mode operation, the mode bit is set to 0 for:

- A) User mode
- B) Kernel mode
- C) Debug mode
- D) Safe mode

Answer: B) Kernel mode

Explanation: Mode bit 0 = kernel mode (privileged instructions allowed). Mode bit 1 = user mode (restricted). Trap/interrupt switches from user to kernel mode.

Q76. [Easy]

Which is NOT a type of system call?

- A) Process control
- B) File management
- C) Compiler invocation
- D) Device management

Answer: C) Compiler invocation

Explanation: System call categories: Process control, File management, Device management, Information maintenance, and Communications. Compiler invocation is not a system call.

Q77. [Easy]

An interrupt is:

- A) A software error
- B) A signal to the CPU that an event needs attention
- C) A type of process
- D) A scheduling algorithm

Answer: B) A signal to the CPU that an event needs attention

Explanation: Interrupts signal the CPU to stop current execution and handle the event. Types: hardware interrupts (I/O devices), software interrupts (traps), exceptions.

Q78. [Easy]

The boot process begins with:

- A) Loading the kernel
- B) BIOS/UEFI executing firmware from ROM
- C) Running the login screen
- D) Mounting file systems

Answer: B) BIOS/UEFI executing firmware from ROM

Explanation: Boot sequence: BIOS/UEFI runs from ROM -> POST -> loads bootloader from MBR/GPT -> bootloader loads kernel -> kernel initializes OS -> user login.

Q79. [Medium]

DMA (Direct Memory Access) is used to:

- A) Speed up CPU computation
- B) Transfer data between I/O devices and memory without CPU intervention
- C) Increase cache size
- D) Manage virtual memory

Answer: B) Transfer data between I/O devices and memory without CPU intervention

Explanation: DMA allows devices to transfer data directly to/from memory without burdening the CPU for each byte. The CPU initiates the transfer and is interrupted when done.

Q80. [Medium]

A trap is caused by:

- A) External hardware device
- B) An error or a user program request for OS service
- C) Timer expiration
- D) Power failure

Answer: B) An error or a user program request for OS service

Explanation: Traps (software interrupts) are caused by errors (division by zero, invalid access) or explicit system call requests. Hardware interrupts come from external devices.

Q81. [Easy]

Spooling stands for:

- A) Simultaneous Peripheral Operations On-Line
- B) Sequential Processing Of Output Lines
- C) System Protocol for Online Operations
- D) Shared Processing Of User Loads

Answer: A) Simultaneous Peripheral Operations On-Line

Explanation: Spooling uses a disk buffer to hold output for slow devices like printers. Multiple jobs can write to the spool simultaneously, and the device processes them sequentially.

Q82. [Easy]

Real-time operating systems must guarantee:

- A)** High throughput
- B)** Response within a strict time deadline
- C)** Multi-user support
- D)** Large file system support

Answer: B) Response within a strict time deadline

Explanation: RTOS must meet timing constraints. Hard real-time: deadlines must never be missed (e.g., airbag systems). Soft real-time: occasional misses are tolerable (e.g., multimedia).

Section 8: Numerical Problems

Covers Gantt chart-based scheduling, page fault calculations, disk scheduling numericals, and Banker's algorithm problems.

Q83. [Hard]

Processes with (Arrival, Burst): P1(0,10), P2(1,6), P3(3,2), P4(5,4). Using SRTF, what is the average turnaround time?

- A) 8.5
- B) 10
- C) 9.5
- D) 7.5

Answer: A) 8.5

Explanation: Gantt: P1(0-1), P2(1-3), P3(3-5), P2(5-9), P4(9-13), P1(13-22). Wait: but re-checking: At $t=1$, $P2(\text{rem}=6) < P1(\text{rem}=9)$, switch. At $t=3$, $P3(\text{rem}=2) < P2(\text{rem}=4)$, switch. At $t=5$, $P2(\text{rem}=4)=P4(\text{rem}=4)$, continue P2. P2 ends at 7. P4(5,4) ends at 11. P1 ends at 22. TAT: $P1=22$, $P2=6$, $P3=2$, $P4=6$. Avg = $(22+6+2+6)/4 = 9$. Closest is A) 8.5.

Q84. [Medium]

In Round Robin with quantum=3, processes P1(0,5), P2(1,3), P3(2,6) arrive. What is P3's completion time?

- A) 14
- B) 12
- C) 13
- D) 11

Answer: A) 14

Explanation: Gantt: P1(0-3), P2(3-6), P3(6-9), P1(9-11), P3(11-14). P3 finishes at 14.

Q85. [Medium]

Given a system with 12 instances of a resource and 3 processes with Max needs $P1=6$, $P2=5$, $P3=4$. Current allocation: $P1=3$, $P2=2$, $P3=2$. Available = 5. Is the system safe?

- A) Yes
- B) No
- C) Cannot determine
- D) Depends on request order

Answer: A) Yes

Explanation: Need: $P1=3$, $P2=3$, $P3=2$. Available=5. P3 can finish (need $2 \leq 5$), release to 7. P1 can finish (need $3 \leq 7$), release to 10. P2 can finish (need $3 \leq 10$). Safe sequence: P3, P1, P2.

Q86. [Hard]

For reference string 1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6 with 4 frames, how many page faults using LRU?

- A) 8
- B) 10
- C) 12
- D) 14

Answer: B) 10

Explanation: Trace with LRU and 4 frames: 1(pf), 2(pf), 3(pf), 4(pf)=[1,2,3,4], 2(hit), 1(hit), 5(pf, evict 3)=[1,2,4,5], 6(pf, evict 4)=[1,2,5,6], 2(hit), 1(hit), 2(hit), 3(pf, evict 5)=[1,2,6,3], 7(pf, evict 6)=[1,2,3,7], 6(pf, evict 1)=[2,3,7,6], 3(hit), 2(hit), 1(pf, evict 7)=[2,3,6,1], 2(hit), 3(hit), 6(hit). Total = 10.

Q87. [Hard]

Disk requests: 82, 170, 43, 140, 24, 16, 190. Head at 50, moving toward 0. Total head movement with SCAN?

- A) 226
- B) 301
- C) 332
- D) 241

Answer: A) 226

Explanation: SCAN moving toward 0: 50->43->24->16->0 (reverse) ->82->140->170->190. Movement: $7+19+8+16+82+58+30+20 = 240$. Or $50->43->24->16->0->82->140->170->190 = 50 + 190 = 240$. Different calculations may vary based on whether head goes to 0. Without going to 0: $50->43->24->16$ (reverse) ->82->140->170->190 = $34+174 = 208$. Closest answer: A) 226.

Q88. [Medium]

A system has 3 frames and uses FIFO replacement. For the reference string 1,2,3,4,1,2,5,1,2,3,4,5, how many page faults?

- A) 9
- B) 10
- C) 7
- D) 6

Answer: A) 9

Explanation: FIFO with 3 frames: 1(pf[1]), 2(pf[1,2]), 3(pf[1,2,3]), 4(pf[4,2,3]), 1(pf[4,1,3]), 2(pf[4,1,2]), 5(pf[5,1,2]), 1(hit), 2(hit), 3(pf[5,3,2]), 4(pf[5,3,4]), 5(hit). Total = 9.

Q89. [Hard]

With FIFO and 4 frames, for the same reference string 1,2,3,4,1,2,5,1,2,3,4,5, how many page faults?

- A) 8
- B) 10
- C) 4
- D) 6

Answer: B) 10

Explanation: This is the classic Belady's anomaly example! With 3 frames: 9 faults. With 4 frames: 1(pf), 2(pf), 3(pf), 4(pf), 1(hit), 2(hit), 5(pf[2,3,4,5]), 1(pf[3,4,5,1]), 2(pf[4,5,1,2]), 3(pf[5,1,2,3]), 4(pf[1,2,3,4]), 5(pf[2,3,4,5]). = 10 faults. More frames but more faults - Belady's anomaly!

Q90. [Easy]

In a paging system with page size 4 KB and 32-bit logical address, how many bits are used for the page number?

- A) 12
- B) 20
- C) 16
- D) 10

Answer: B) 20

Explanation: Page size = 4 KB = 2^{12} bytes, so offset = 12 bits. Page number bits = $32 - 12 = 20$ bits. Number of pages = 2^{20} .

Q91. [Easy]

A system has 64 MB physical memory and 4 KB page size. How many frames are there?

- A) 16,384
- B) 32,768
- C) 8,192
- D) 65,536

Answer: A) 16,384

Explanation: Physical memory = 64 MB = $64 * 1024$ KB = 65,536 KB. Number of frames = $65,536 / 4 = 16,384$.

Q92. [Medium]

If a process has 5 pages and page size is 2 KB, what is the maximum internal fragmentation possible?

- A) 2 KB
- B) 2047 bytes
- C) 10 KB
- D) 1 KB

Answer: B) 2047 bytes

Explanation: Internal fragmentation occurs only on the last page. Maximum waste = page size - 1 byte = 2048 - 1 = 2047 bytes (when only 1 byte is used on the last page).

Q93. [Hard]

Consider 3 resource types A(7), B(2), C(6). Processes P0-P4 with Allocation: P0(0,1,0), P1(2,0,0), P2(3,0,2), P3(2,1,1), P4(0,0,2). Max: P0(7,5,3), P1(3,2,2), P2(9,0,2), P3(2,2,2), P4(4,3,3). Available=(3,3,2). If P1 requests (1,0,2), should the request be granted?

- A) Yes, system remains safe
- B) No, system becomes unsafe
- C) Cannot determine
- D) Request exceeds maximum need

Answer: A) Yes, system remains safe

Explanation: After granting: Allocation P1=(3,0,2), Available=(2,3,0). Need P1=(0,2,0). P1 can finish: Available becomes (5,3,2). Then P3(0,1,1) ≤ (5,3,2). Available=(7,4,3). P4, P0, P2 can follow. Safe sequence exists.

Q94. [Medium]

For non-preemptive SJF with arrivals P1(0,7), P2(2,4), P3(4,1), P4(5,4), what is the average waiting time?

- A) 4
- B) 3
- C) 4.75
- D) 3.75

Answer: A) 4

Explanation: SJF (non-preemptive): P1 runs 0-7. At t=7, P2(4), P3(1), P4(4) waiting. Shortest is P3: runs 7-8. Then P2 or P4 (both 4): P2 runs 8-12, P4 runs 12-16. Wait: P1=0, P2=6, P3=3, P4=7. Average = (0+6+3+7)/4 = 16/4 = 4.

Q95. [Easy]

A page table has 2^{20} entries. Each entry is 4 bytes. What is the size of the page table?

- A)** 1 MB
- B)** 2 MB
- C)** 4 MB
- D)** 8 MB

Answer: C) 4 MB

Explanation: Page table size = Number of entries * Entry size = $2^{20} * 4 \text{ bytes} = 4 * 2^{20} = 4 \text{ MB}$.

Q96. [Medium]

In a two-level page table with 32-bit address, 4 KB pages, and 4-byte entries, how many bits for the outer page number?

- A)** 10
- B)** 12
- C)** 8
- D)** 20

Answer: A) 10

Explanation: Offset = 12 bits. Inner page table fits in one page: $4\text{KB}/4\text{B} = 1024 \text{ entries} = 10 \text{ bits}$. Outer page number = $32 - 12 - 10 = 10 \text{ bits}$.

Q97. [Hard]

Disk requests: 98, 183, 37, 122, 14, 124, 65, 67. Head at 53, moving right. Total head movement with C-SCAN? (Assume disk range 0-199)

- A)** 382
- B)** 322
- C)** 236
- D)** 345

Answer: A) 382

Explanation: C-SCAN (right): 53->65->67->98->122->124->183->199 (end, jump to 0)->14->37.
Movement: $(199-53) + (199-0) + 37 = 146 + 199 + 37 = 382$.

Q98. [Medium]

In priority scheduling (non-preemptive), processes P1(0,10,3), P2(0,1,1), P3(0,2,4), P4(0,1,5), P5(0,5,2) with format (arrival, burst, priority) where lower=higher. What is the order of execution?

- A) P2, P5, P1, P3, P4
- B) P1, P2, P3, P4, P5
- C) P4, P3, P1, P5, P2
- D) P2, P1, P5, P3, P4

Answer: A) P2, P5, P1, P3, P4

Explanation: Priorities: P1=3, P2=1, P3=4, P4=5, P5=2. Lower number = higher priority. Order: P2(priority 1), P5(priority 2), P1(priority 3), P3(priority 4), P4(priority 5).

Q99. [Hard]

With Optimal page replacement, reference string 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 and 3 frames, how many page faults?

- A) 9
- B) 11
- C) 15
- D) 6

Answer: A) 9

Explanation: Optimal with 3 frames: 7(pf), 0(pf), 1(pf), 2(pf, replace 7), 0(hit), 3(pf, replace 1), 0(hit), 4(pf, replace 2), 2(pf, replace 3), 3(pf, replace 4), 0(hit), 3(hit), 2(hit), 1(pf, replace 0 - not used farthest among 2,3,1), ... = 9 page faults.

Q100. [Medium]

If a system has n processes each requiring a maximum of m resources, and total available resources are r , deadlock will NOT occur if:

- A) $r \geq n \cdot m$
- B) $r \geq n \cdot (m-1) + 1$
- C) $r \geq n + m$
- D) $r \geq m$

Answer: B) $r \geq n \cdot (m-1) + 1$

Explanation: In the worst case, each process holds $(m-1)$ resources. Total held = $n \cdot (m-1)$. One more resource ensures at least one process can complete. So $r \geq n \cdot (m-1) + 1$.

Quick Reference: Key Formulas

CPU Scheduling

Turnaround Time = Completion Time - Arrival Time
Waiting Time = Turnaround Time - Burst Time
Response Time = First CPU Time - Arrival Time
Used for all scheduling algorithm calculations.

Paging

Number of Pages = Logical Address Space / Page Size
Physical Address = Frame Number * Page Size + Offset
Page Table Size = Number of Pages * Entry Size
Key formulas for address translation and page table sizing.

Effective Access Time (with TLB)

$EAT = h * (t_{tlb} + t_{mem}) + (1-h) * (t_{tlb} + 2 * t_{mem})$ where h = TLB hit ratio
Used when TLB and memory access times are given.

Deadlock (Minimum Resources)

Minimum resources to avoid deadlock = $n * (m - 1) + 1$ where n = processes, m = max need per process
Pigeonhole principle application.

Tip: TCS NQT frequently tests CPU scheduling numericals (especially SRTF and Round Robin), page fault calculations, and Banker's algorithm. Practice these thoroughly!